

BHARATI VIDYAPEETH'S COLLEGE OF ENGINEERING

A-4, PASCHIM VIHAR, NEW DELHI - 110063

CERTIFICATE

This is to Certify that Mr./Miss... FARAZ KHAN.....

Roll No.

0	5	7	1	1	5	0	2	7	2	1
---	---	---	---	---	---	---	---	---	---	---

 Branch:..... CSE-1.....

has satisfactorily completed his / her terms work in

Digital Logic and Computer Design - Lab.....subject

Head of the Department

Date

Professor
Incharge

EXPERIMENT - 1

AIM : Implementation of Adder circuit in Logisim.

REQUIREMENTS :

THEORY :

Adder : In electronics an adder is digital circuit that performs addition of numbers. In Modern computers adder resides in the Arithmetic Logic Unit (ALU).

- Adders are important not only in the computer but also in many types of digital systems in which the numeric data are processed.

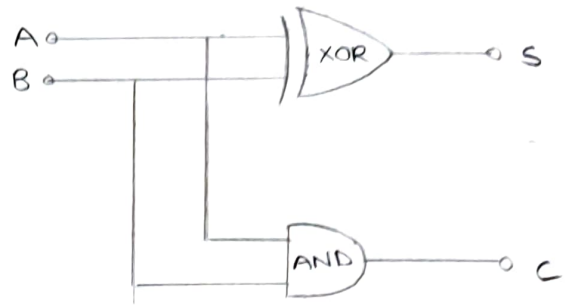
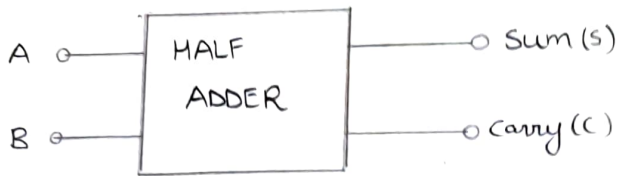
Types of Adder

- 1) Half Adder : The half adder accepts two binary digits on its inputs and produce two binary digits outputs, a sum bit and a carry bit.
- The half adder is an example of a simple, functional digital circuit built from two logic gates. The half adder adds to one-bit binary numbers (AB). The output is the sum of the two bits sum (S) and the carry (C).
- The same two inputs are directed to two different gates. The inputs to the XOR gate are also the inputs to the AND gate. The input "wires" to the XOR gate are tied to the input wires of the AND gate; thus, when voltage is applied to the A input of the XOR gate, the A input to the AND gate receives the same voltage.

The expressions obtained are :

CIRCUIT DIAGRAM :

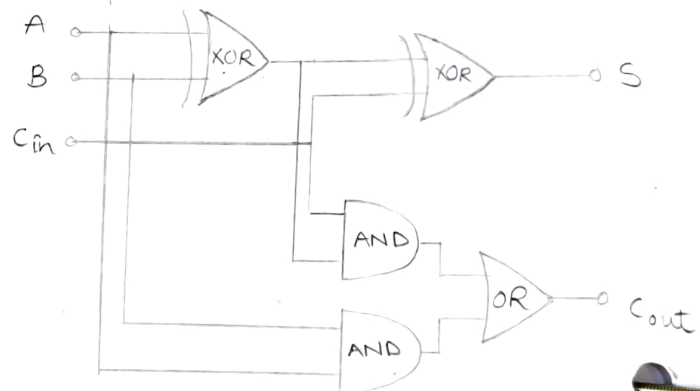
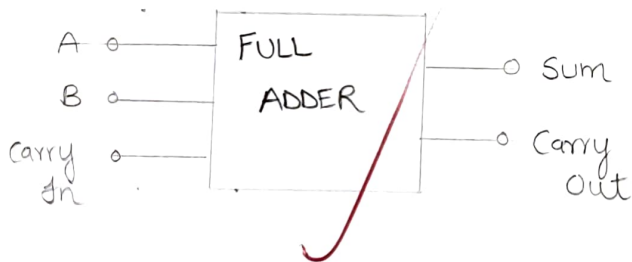
1) HALF ADDER :



TRUTH
TABLE :

A	B	Sum	Carry Out
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

2) FULL ADDER :



TRUTH
TABLE :

A	B	Carry In	Sum	Carry Out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$S = A \oplus B$$

$$C = A \cdot B$$

2.) Full Adder : The full adder accepts two inputs bits and an input carry and generates a sum output and an output carry.

- The full adder adds three one-bit binary numbers (A, B, C_{in}) and outputs two one-bit binary numbers, a sum (S) and a carry (C_{out}). The full adder is usually a component in a cascade of adders, which adds 8, 16, 32 etc binary numbers.
- The full adder is simply two half adders joined by an OR.
- We can implement a full adder circuit with help of two half adder circuits. The first half adder will be used to add A and B to produce a partial sum. The second half adder logic can be used to add C_{in} to the sum produced by the first half adder to get the final S output. If any of the adder logic produces a carry, there will be an output carry, thus C_{out} will be an OR function of the half adder carry outputs.

The expressions obtained are :

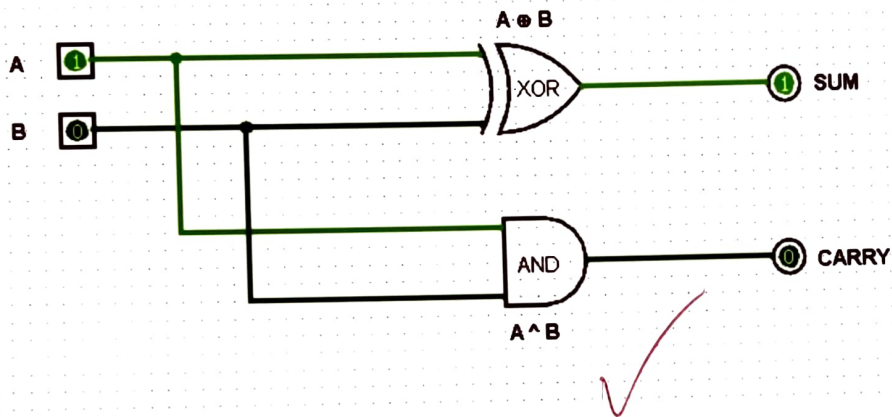
$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + BC_{in} + C_{in}A$$

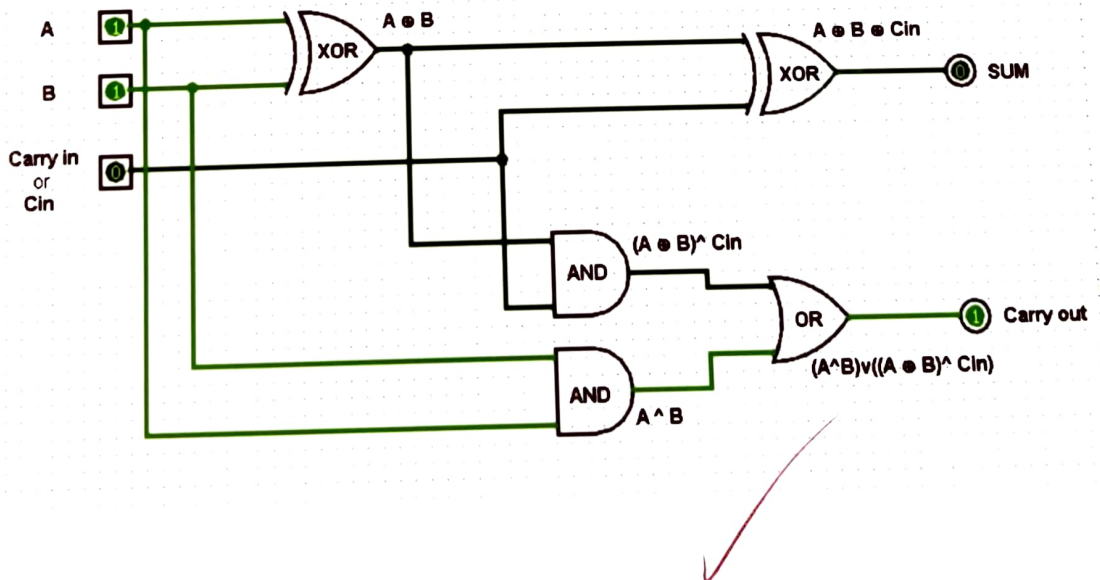
RESULT : The circuit of Half adder and Full adder have been studied and their truth tables were verified.

OUTPUT :

HALF ADDER

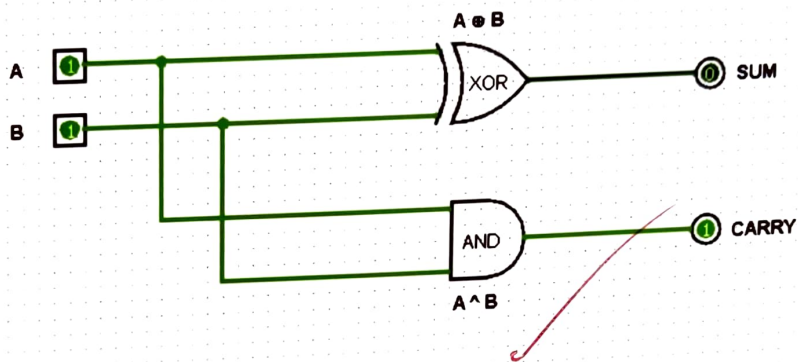


FULL ADDER

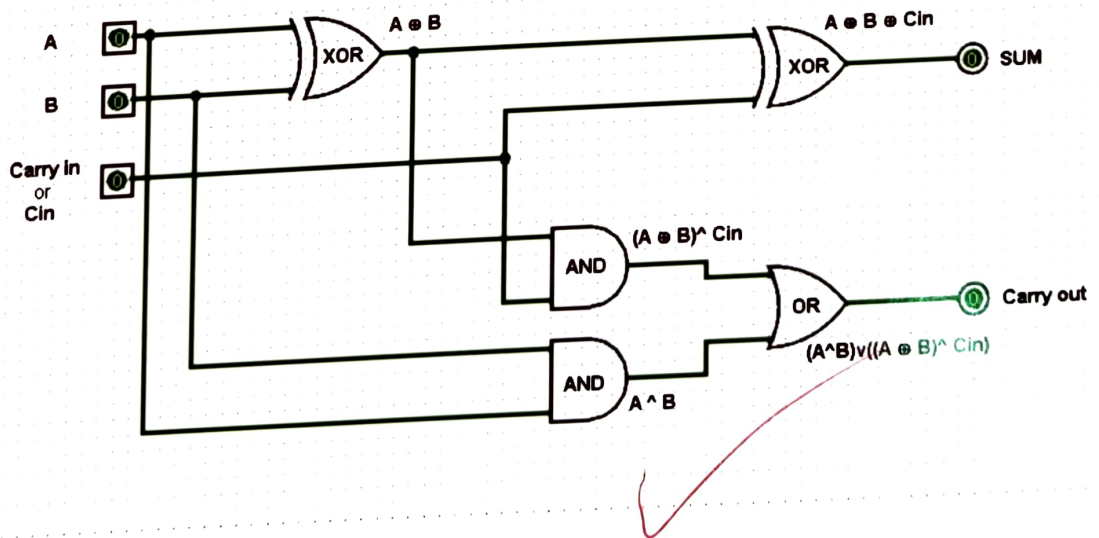


OUTPUT :

HALF ADDER



FULL ADDER



EXPERIMENT-2

AIM : To design and implement binary parallel adder circuit in Logisim.

REQUIREMENTS : Logisim software.

THEORY :

Parallel Adder :

- It is a circuit consisting of n -full adders, that will add n -bit binary numbers.
- The output consists of n sum bits and a carry bit.
- Output of one full adder is connected to input of the next full adder.

Structure of Parallel Adder :

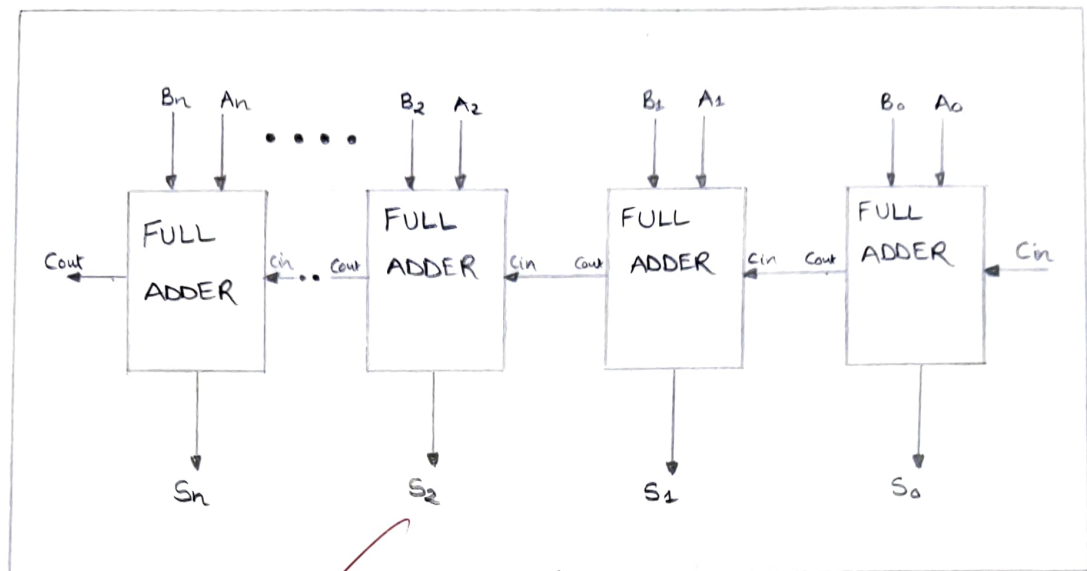
- A parallel adder is nothing but a cascade of several full adder.
- The number of full adders used will depend on the number of bits in the binary digits which require to be added.

Block diagram of n -bit binary parallel adder :

When a n -bit binary number is added to another, each column generates a sum and a 0 or 1 carry to the next higher order column.

PROCEDURE :

- The bits are added with full adder.
- Starting from the least significant position (LSB) to form the sum and carry.
- The input carry C_{in} in the least significant position must be zero.
- The value of C_{out} is a given significant position is the output carry C_{out} of the full adder.



BLOCK DIAGRAM OF N-bit BINARY PARALLEL ADDER

Demonstration :

For eg : $A = 1010$, $B = 1011$

$$\begin{array}{r}
 A+B = 1010 \\
 + 1011 \\
 \hline
 \text{Sum} = 10101
 \end{array}$$

SUBSCRIPT	3	2	1	0	
Input carry	0	1	0	0	C_i
Augend (A)	1	0	1	0	A_i
Addend (B)	1	0	1	1	B_i
Sum	0	1	0	1	S_i
Output carry	1	0	1	0	C_{i+1}

adder that adds the bits one higher significant position to the left.

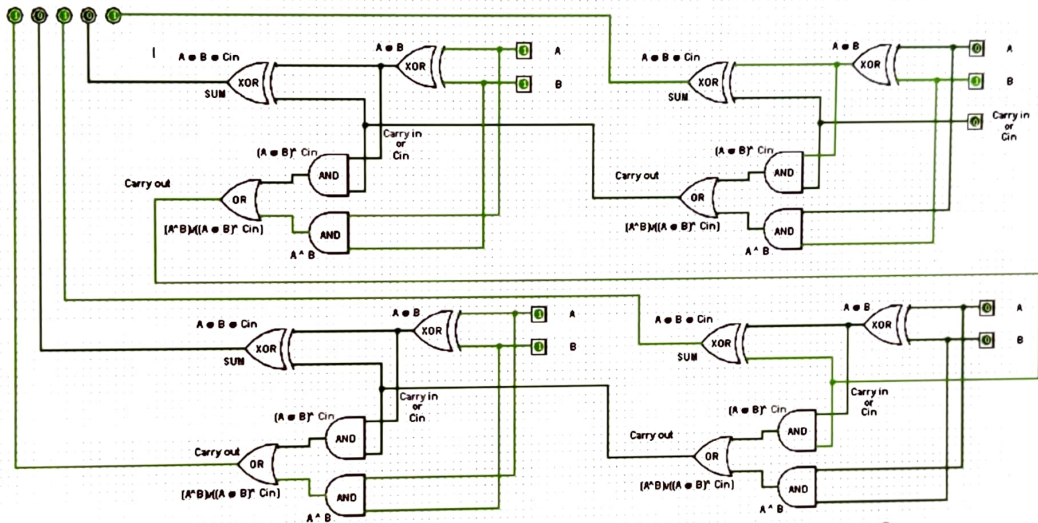
- The sum bits are thus generated starting from the right most position and are available as soon as the corresponding previous carry bit is generated.

RESULT :

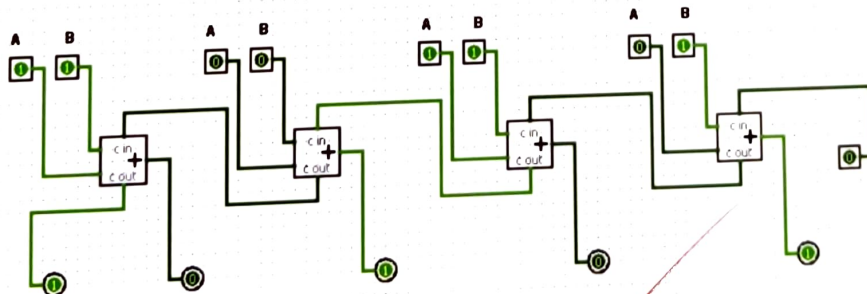
The circuit of a 4-bit binary Adder has been studied and its output has been verified.

OUTPUT:

4-bit Binary Adder



4-bit Binary Adder



EXPERIMENT - 3

Aim : To implement a subtractor circuit in Logisim.

REQUIREMENTS : Logisim software.

THEORY :

Subtractor : In electronics a subtractor is digital circuit that performs subtraction of numbers. Subtractors are used in electronic calculators as well as digital devices.

Types of subtractor :

- 1.) Half Subtractor : It is used for subtracting one single bit binary number from another single bit binary number. It takes two inputs a Minuend (A) and Subtrahend (B) and two outputs i.e. Difference (D) and Borrow (Bout). The half subtractor is designed with the help of the following logic gates : i) 2-input AND gate ii) 2-input Exclusive-OR gate or XOR gate and iii) NOT gate. The difference bit (D) is generated with the help of the XOR gate and the Borrow bit (Bout) is generated with the help of a NOT and AND gate.

Boolean expression : from the truth table and K-map,

the boolean expression can be derived as :

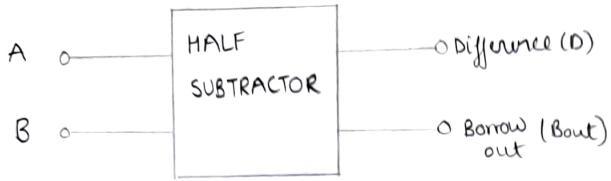
$$\text{Difference (D)} = \bar{A} \cdot B + A \cdot \bar{B} = A \oplus B$$

$$\text{Borrow (Bout)} = \bar{A} \cdot B$$

- 2.) Full Subtractor : It is a combinational circuit that performs subtraction of two single bit number. It takes three inputs a Minuend (A), subtrahend (B) and previous

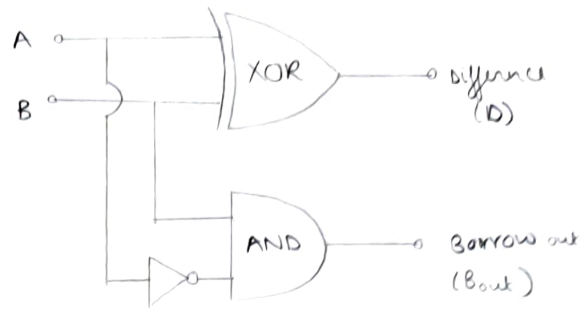
CIRCUIT DIAGRAM :

1.) HALF SUBTRACTOR :

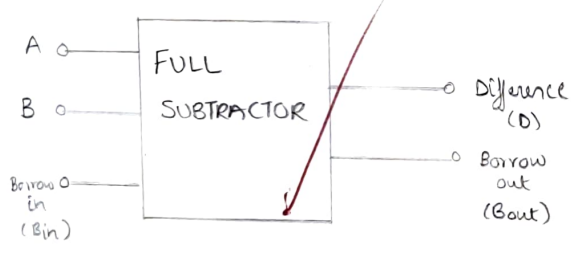


TRUTH TABLE :

A	B	Difference (D)	Borrow out (Bout)
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

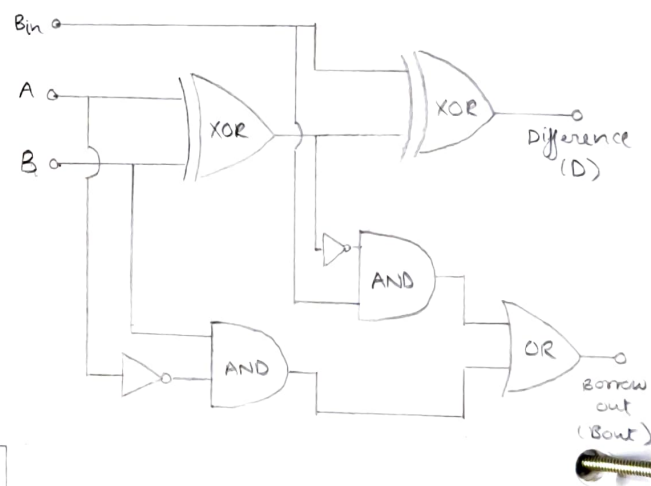


2.) FULL SUBTRACTOR :



TRUTH TABLE :

A	B	Bin	Difference (D)	Borrow out (Bout)
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1



borrow (B_{in}).

The full subtractor is designed with the help of the following logic gates : i) two 2-input AND gate ii) two 2-input Exclusive -OR gate or XOR gate iii) two NOT gate iv) 2-input OR gate.

Full subtractor generates two outputs a Difference bit (D) and a Borrow bit (B_{out}).

Boolean Expression : from the truth table and K-map, the boolean expression can be derived as :

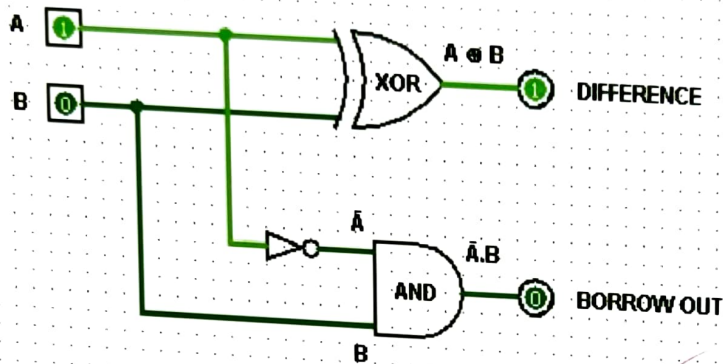
$$\begin{aligned}\text{Difference (D)} &= \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}\bar{C} + ABC \quad \text{where } C = B_{in} \\ &= A \oplus B \oplus C \quad \equiv A \oplus B \oplus B_{in}\end{aligned}$$

$$\begin{aligned}\text{Borrow (Bout)} &= \bar{A}\bar{B}C + \bar{A}B\bar{C} + \bar{A}BC + ABC \\ &= \bar{A}C(B + \bar{B}) + \bar{A}B(C + \bar{C}) + BC(A + \bar{A}) \\ &= \bar{A}C + \bar{A}B + BC \quad \text{where } C = B_{in} \\ &= \bar{A}B_{in} + \bar{A}B + BB_{in} \\ &\equiv BC + (B \oplus C)A\end{aligned}$$

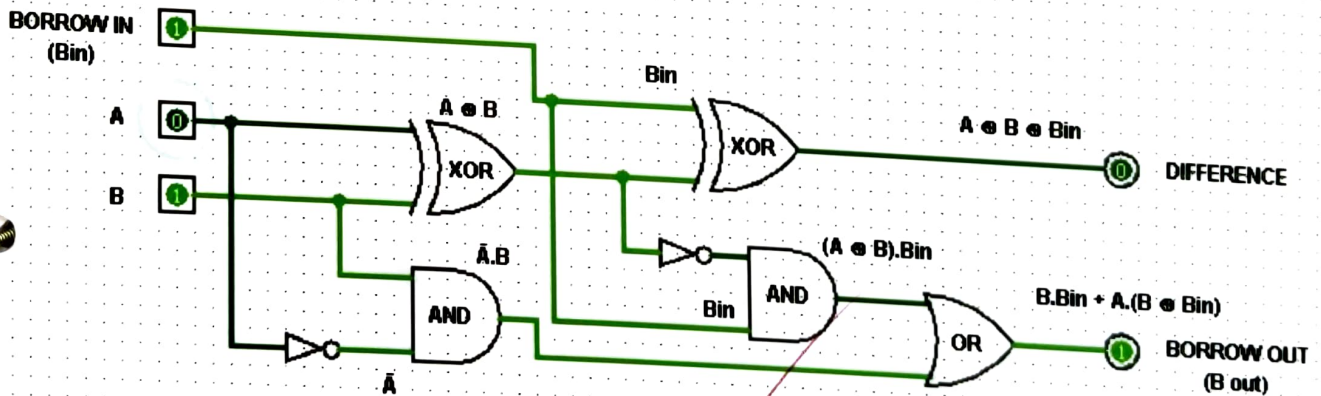
RESULT : The circuits of Half subtractor and Full subtractor have been studied and their truth tables were verified.

OUTPUT :

HALF SUBTRACTOR



FULL SUBTRACTOR



EXPERIMENT - 4

Aim : Design and Implementation of Multiplexer and De-Multiplexer in Logisim.

REQUIREMENTS : Logisim Software

THEORY :

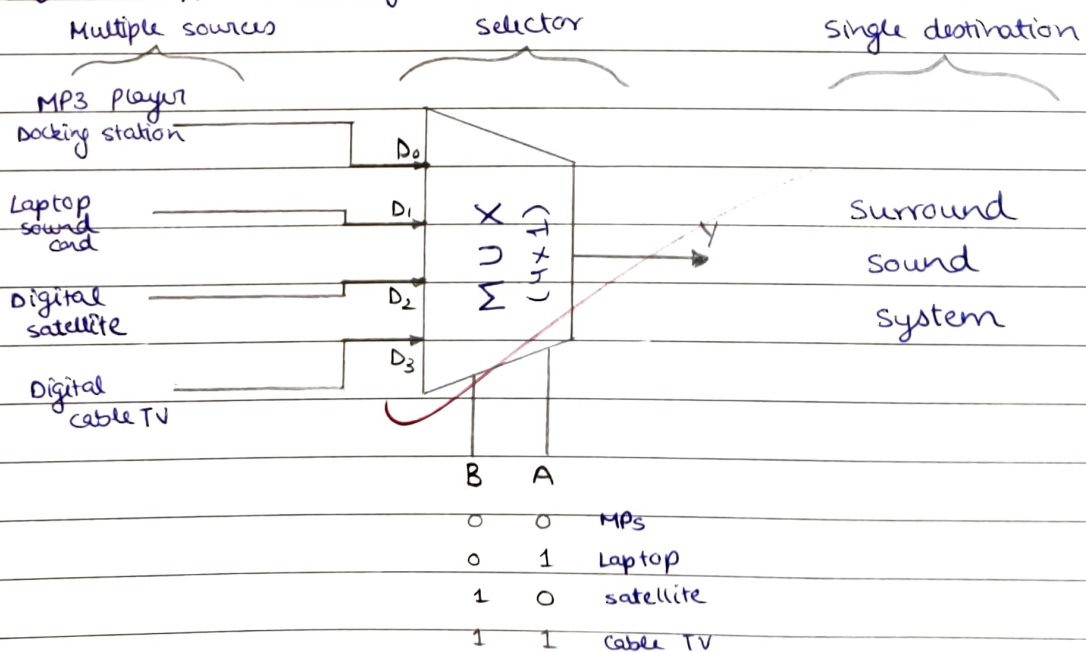
Multiplexer (MUX) :

- A MUX is a digital switch that has multiple inputs (sources) and a single output (destination)
- The select lines determine which input is connected to the output.
- MUX Types :

- (i) 2 to 1 (1 select line)
- (ii) 4 to 1 (2 select lines)
- (iii) 8 to 1 (3 select lines)
- (iv) 16 to 1 (4 select lines)

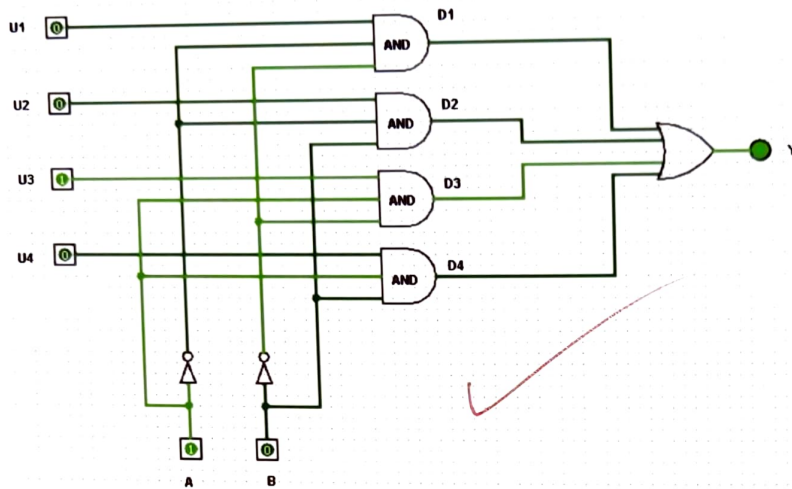
In a generalized $N \times 1$ MUX where N is 2^m has 'm' select lines.

Typical Application of a MUX :

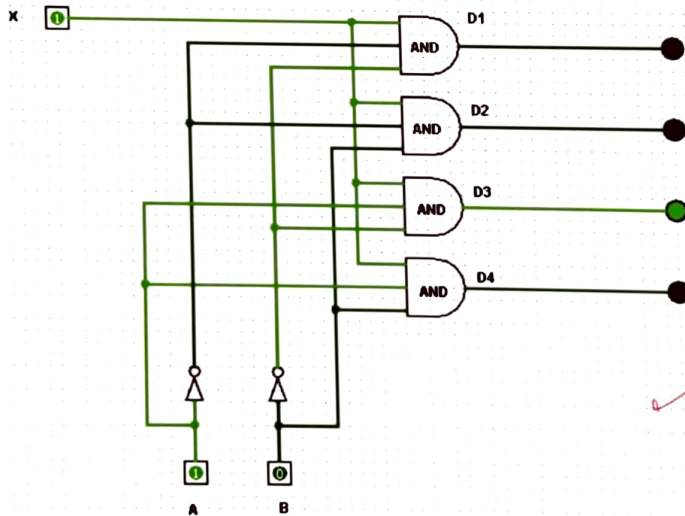


OUTPUT:

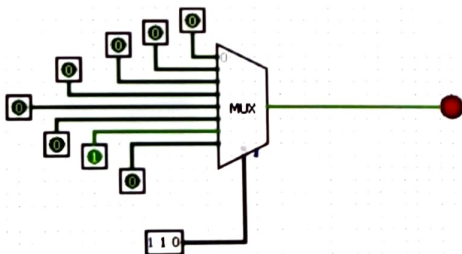
MULTIPLEXER (4 X 1)



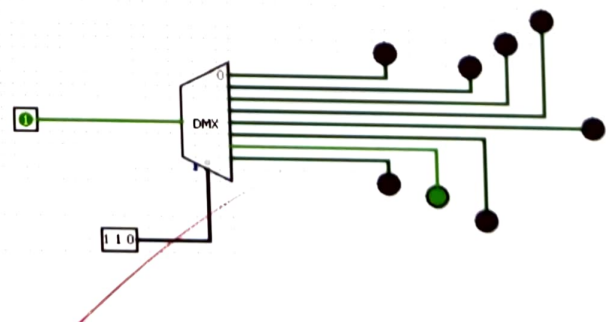
DE-MULTIPLEXER (1 X 4)



MULTIPLEXER (8 X 1)



DE-MULTIPLEXER (1 X 8)



EXPERIMENT - 5

AIM : Design BCD to 7 segment LED Display Decoder using Logisim.

REQUIREMENTS : Logisim software

THEORY :

-) In Binary Coded Decimal (BCD) encoding scheme each of the decimal numbers (0-9) is represented by its equivalent binary pattern (which is generally of 4 bits).
-) whereas, Seven segment display is an electronic device which consists of seven Light Emitting Diodes (LED's) arranged in a same definite pattern which is used to display numbers 0 to 9.
-) BCD to seven segment decoder has four input lines (A, B, C and D) and 7 output lines (a, b, c, d, e, f and g), thus output is given to seven segment LED display which displays the decimal number depending upon inputs.
-) Seven segment displays are used to display the digits in calculators, clocks, various measuring instruments, digital watches and digital counters.
-) From the Truth Table, the Boolean expressions of each output functions can be written as :

$$a = F_1(A, B, C, D) = \sum m(0, 2, 3, 5, 7, 8, 9)$$

$$b = F_2(A, B, C, D) = \sum m(0, 1, 2, 3, 4, 7, 8, 9)$$

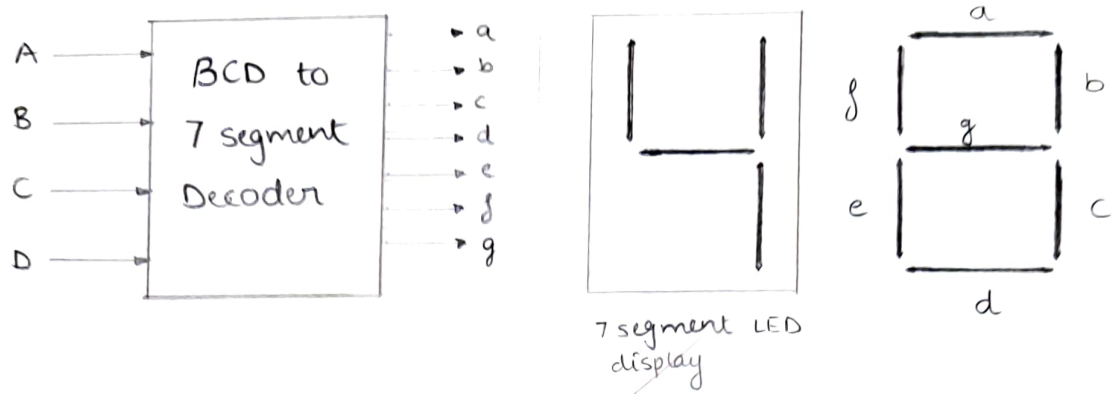
$$c = F_3(A, B, C, D) = \sum m(0, 1, 3, 4, 5, 6, 7, 8, 9)$$

$$d = F_4(A, B, C, D) = \sum m(0, 2, 3, 5, 6, 8)$$

$$e = F_5(A, B, C, D) = \sum m(0, 2, 6, 8)$$

$$f = F_6(A, B, C, D) = \sum m(0, 4, 5, 6, 8, 9)$$

$$g = F_7(A, B, C, D) = \sum m(2, 3, 4, 5, 6, 8, 9)$$



TRUTH TABLE :

Decimal Digit	Input lines				Output lines							Display pattern
	A	B	C	D	a	b	c	d	e	f	g	
0	0	0	0	0	1	1	1	1	1	1	0	
1	0	0	0	1	0	1	1	0	0	0	0	
2	0	0	1	0	1	1	0	1	1	0	1	
3	0	0	1	1	1	1	1	1	0	0	1	
4	0	1	0	0	0	1	1	0	0	1	1	
5	0	1	0	1	1	0	1	1	0	1	1	
6	0	1	1	0	1	0	1	1	1	1	1	
7	0	1	1	1	1	1	1	0	0	0	0	
8	1	0	0	0	1	1	1	1	1	1	1	
9	1	0	0	1	1	1	1	1	0	1	1	

-) From the above simplification, we get the output value as :

$$a = A + C + BD + \overline{B}\overline{D}$$

$$b = \overline{B} + \overline{C}\overline{D} + CD$$

$$c = B + \overline{C} + D$$

$$d = \overline{B}\overline{D} + C\overline{D} + B\overline{C}D + \overline{B}C + A$$

$$e = \overline{B}\overline{D} + C\overline{D}$$

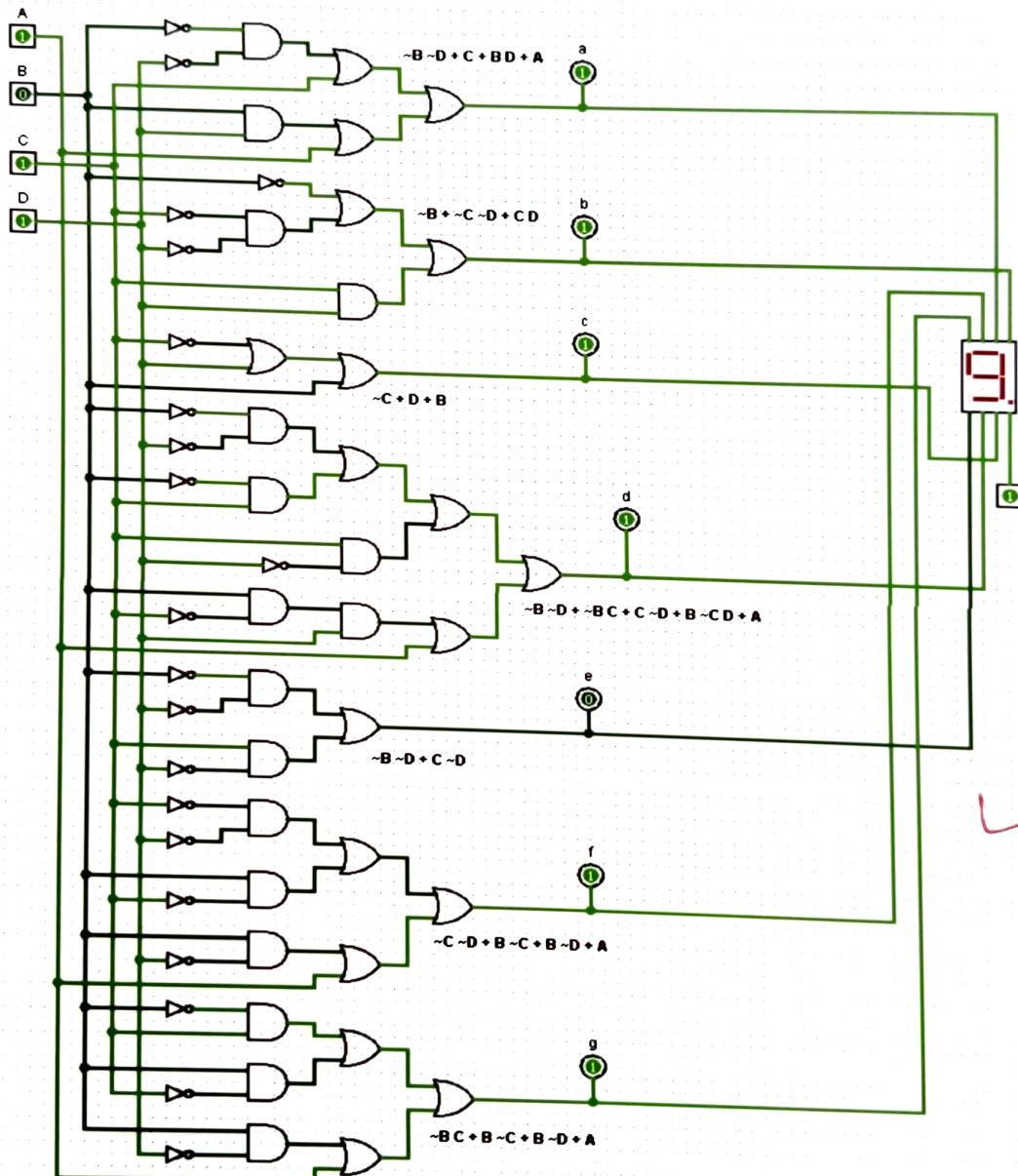
$$f = A + \overline{C}\overline{D} + B\overline{C} + B\overline{D}$$

$$g = A + B\overline{C} + \overline{B}C + C\overline{D}$$

RESULT : The circuit of BCD to 7 segment decoder has been studied and its output has been verified.

OUTPUT :

BCD To 7 SEGMENT DECODER



EXPERIMENT - 6

AIM : Design 4-bit Ripple Counter in Logisim :

- Up Counter
- Down Counter
- MOD - 12 Counter

REQUIREMENTS : Logisim Software

THEORY :

i) **Ripple Counter :**

Ripple Counter is a special type of asynchronous counter in which the clock pulse ripples through the circuit. In a ripple counter, the flip-flop output transition serves as a source of triggering other flip-flops.

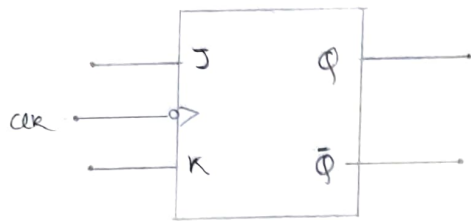
ii) **Asynchronous Counter:**

Asynchronous counters are also called ripple counters because of the way the clock pulse ripples its way through the flip-flops. Counters are of two types depending upon the clock pulse applied. These counters are Asynchronous and Synchronous counters.

iii) **Synchronous Counter:**

In Synchronous counter, all flip-flops are triggered with same clock simultaneously. In asynchronous counter, different flip-flops are triggered with different clocks, not simultaneously.

• Negative edge Triggered Flip Flop :



TRUTH TABLE

CLK	J	K	Q_{n+1}
↓	0	0	Q_n
↓	0	1	0
↓	1	0	1
↓	1	1	Q_n

4-BIT RIPPLE UP - COUNTER

TRUTH TABLE :

CLK	Q_D	Q_C	Q_B	Q_A	Decimal
Initial	0	0	0	0	0
1 st (↓)	0	0	0	1	1
2 nd (↓)	0	0	1	0	2
3 rd (↓)	0	0	1	1	3
4 th (↓)	0	1	0	0	4
5 th (↓)	0	1	0	1	5
6 th (↓)	0	1	1	0	6
7 th (↓)	0	1	1	1	7
8 th (↓)	1	0	0	0	8
9 th (↓)	1	0	0	1	9
10 th (↓)	1	0	1	0	10
11 th (↓)	1	0	1	1	11
12 th (↓)	1	1	0	0	12
13 th (↓)	1	1	0	1	13
14 th (↓)	1	1	1	0	14
15 th (↓)	1	1	1	1	15

4-BIT RIPPLE DOWN - COUNTER

TRUTH TABLE :

CLK	$\overline{Q_D}$	$\overline{Q_C}$	$\overline{Q_B}$	$\overline{Q_A}$	Decimal
Initial	1	1	1	1	15
1 st (↓)	1	1	1	0	14
2 nd (↓)	1	1	0	1	13
3 rd (↓)	1	1	0	0	12
4 th (↓)	1	0	1	1	11
5 th (↓)	1	0	1	0	10
6 th (↓)	1	0	0	1	9
7 th (↓)	1	0	0	0	8
8 th (↓)	0	1	1	1	7
9 th (↓)	0	1	1	0	6
10 th (↓)	0	1	0	1	5
11 th (↓)	0	1	0	0	4
12 th (↓)	0	0	1	1	3
13 th (↓)	0	0	1	0	2
14 th (↓)	0	0	0	1	1
15 th (↓)	0	0	0	0	0

iv) Up Counter :

This counter counts from zero to the maximum number value.

v) Down Counter :

This type of counter counts from the maximum value to zero value.

vi) MOD Counter :

MOD counters are defined based on the number of states that the counter will sequence through before returning back to its original value.

A Modulus - 12 counter will count from 0 (0000) to 11 (1011) and would require four flip-flops.

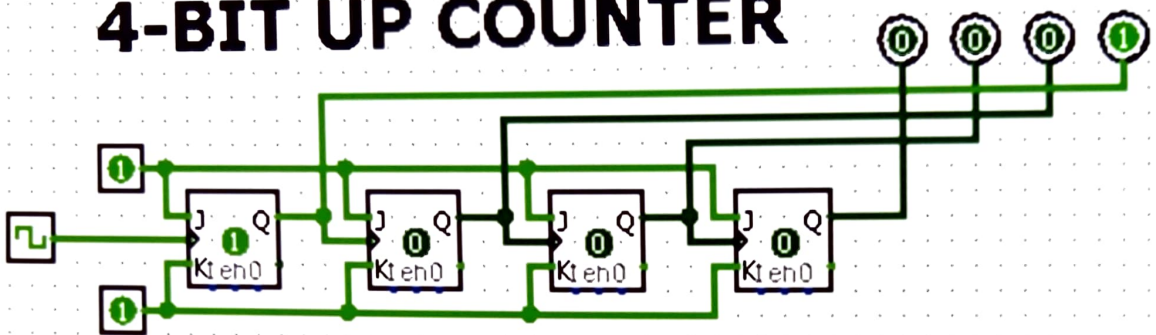
MOD - 12 UP COUNTER

TRUTH TABLE :

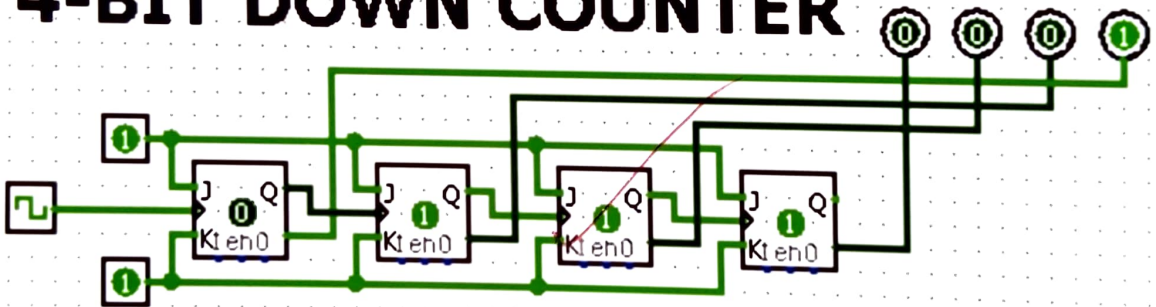
CLK	Φ_D	Φ_C	Φ_B	Φ_A	Decimal
Initial	0	0	0	0	0
1 st ($\downarrow$)	0	0	0	1	1
2 nd ($\downarrow$)	0	0	1	0	2
3 rd ($\downarrow$)	0	0	1	1	3
4 th ($\downarrow$)	0	1	0	0	4
5 th ($\downarrow$)	0	1	0	1	5
6 th ($\downarrow$)	0	1	1	0	6
7 th ($\downarrow$)	0	1	1	1	7
8 th ($\downarrow$)	1	0	0	0	8
9 th ($\downarrow$)	1	0	0	1	9
10 th ($\downarrow$)	1	0	1	0	10
11 th ($\downarrow$)	1	0	1	1	11
12 th ($\downarrow$)	0	0	0	0	0

OUTPUT:

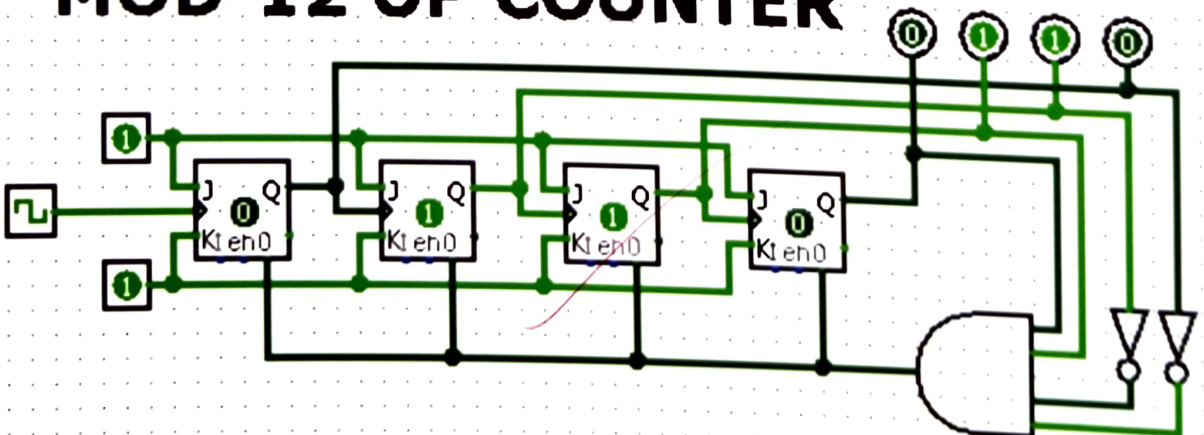
4-BIT UP COUNTER



4-BIT DOWN COUNTER



MOD-12 UP COUNTER



EXPERIMENT - 7

AIM : Implementation of 3-bit ripple counter (Synchronous) on Logisim.

SOFTWARE USED : Logisim

THEORY : Synchronous generally is type of counter in which the clock signal is simultaneously provided to each flip flop present in the counter circuit. More specifically, we can say that each flip flop is triggered in synchronous with the clock input.

→ Steps to design Synchronous Counter

1. Decide the number of flip flops
2. Excitation table of flip flops
3. State diagram and circuit excitation table
4. Obtain simplified eqⁿ using K' map
5. Draw the logic Diagram.

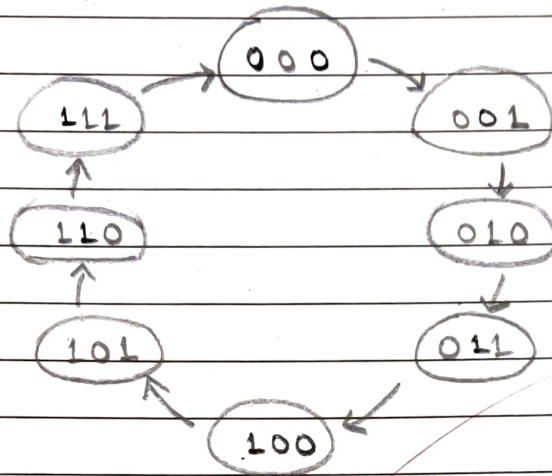
Design 3-bit Synchronous up Counter

1) Excitation table of JK flip flop.

Q_n	Q_{n+1}	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

2)

State diagram and Circuit Excitation table is



Circuit Excitation table:-

Present State			Next State								
Q_2	Q_1	Q_0	Q_2^*	Q_1^*	Q_0^*	J_2	K_2	J_1	K_1	J_0	K_0
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	0	0	X	1	X	X	1
0	1	0	0	1	1	0	X	X	0	1	X
0	1	1	1	0	0	1	X	X	1	X	1
1	0	0	1	0	1	X	0	0	X	1	X
1	0	1	1	1	0	X	0	1	X	X	1
1	1	0	1	1	1	X	0	X	0	1	X
1	1	1	0	0	0	X	1	X	1	X	1

~~K-map for Excitation Table~~For J_2

$Q_2 \backslash Q_1 Q_0$	00	01	11	10
0			1	
1	X	X	X	X

For K_2

$Q_2 \backslash Q_1 Q_0$	00	01	11	10
0	X	X	X	X
1			1	

For J_1

$Q_2 \backslash Q_1 Q_0$	00	01	11	10
0		1	X	X
1	X	X	X	X

$$J_2 = Q_1 Q_0$$

$$K_2 = Q_1 Q_0$$

$$J_1 = Q_0$$

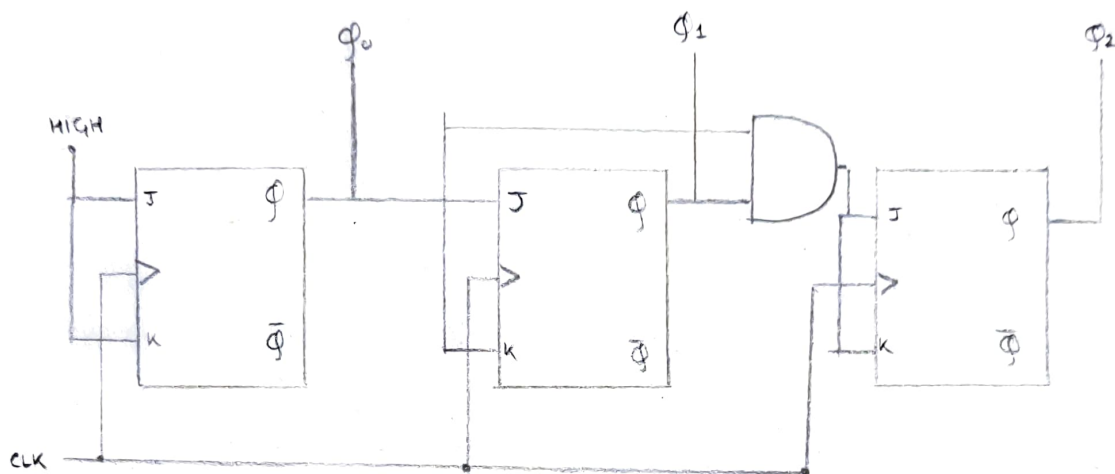


Fig. 3 BIT SYNCHRONOUS COUNTER

For K_1

$Q_2 \mid Q_1 Q_0$					
X_0	X_1	1_3	2		
X_4	X_5	1_7	6		

$$K_1 = Q_0$$

For J_0

$Q_2 \mid Q_1 Q_0$					
1_0	X_1	X_3	1_2		
1_4	X_5	X_7	1_6		

$$J_0 = 1$$

For K_0

$Q_2 \mid Q_1 Q_0$					
X_0	1_1	1_3	X_2		
X_4	1_5	1_7	X_6		

$$K_0 = 1$$

From the previous simplification we get the output as follows,

$$J_0 = 1$$

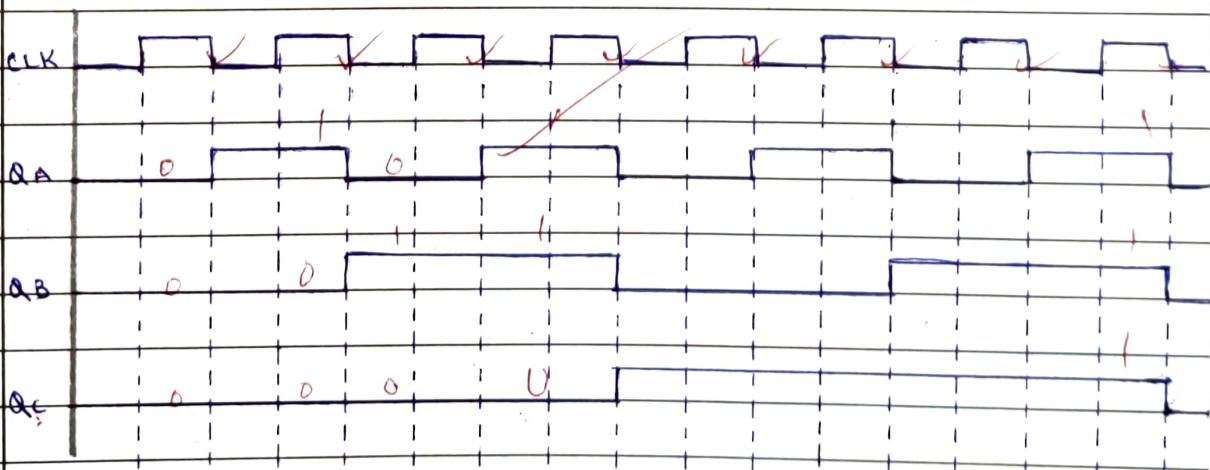
$$J_2 = Q_1 Q_0$$

$$K_0 = 1$$

$$K_2 = Q_1 Q_0$$

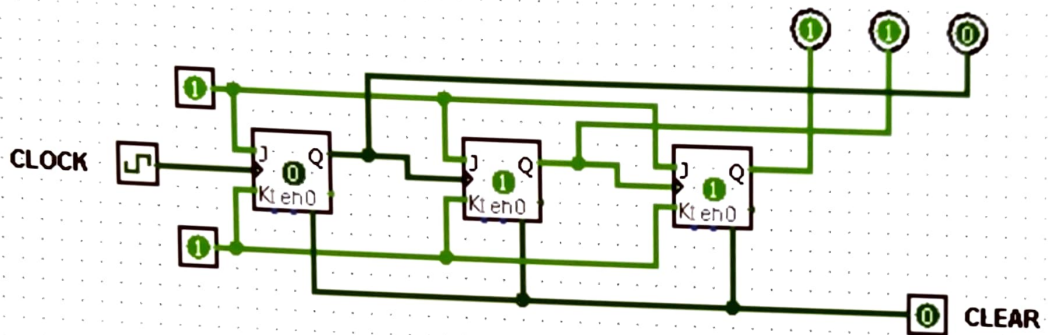
$$J_1 = Q_0$$

$$K_1 = Q_0$$

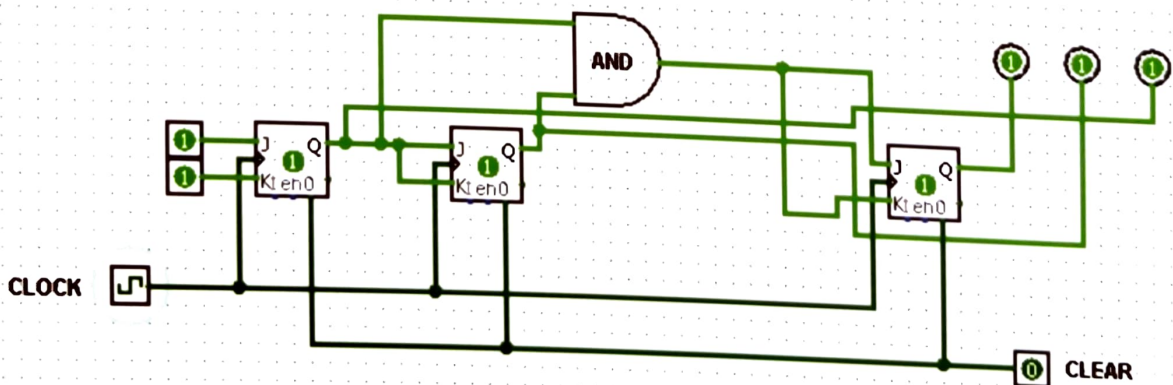


OUTPUT:

3 - BIT ASYNCHRONOUS COUNTER



3 - BIT SYNCHRONOUS COUNTER



EXPERIMENT - 8

AIM : Writing and executing programs in gnuSim 8085.

SOFTWARE USED : GNU Sim 8085

THEORY :

- GNU Sim 8085 is a software that simulates the 8085 microprocessor. It can assemble, debug and execute 8085 assembly code.
- There are 6 general purpose registers to store data, namely B, C, D, E, H, L and an Accumulator to store the result.
- We can convert decimal to hex and vice versa.
- We can also manually update port values and memory.
- Some common commands

MVI $\Rightarrow$ Move Immediately

MOV $\Rightarrow$ store data at a particular address

LDA $\Rightarrow$ Load Accumulator

ADD $\Rightarrow$ Add to the result in Accumulator

STA $\Rightarrow$ Copy Accumulator contents at address

HLT $\Rightarrow$ Halt.

PROGRAMS :

→ To Add two values

- 1) MVI B, 05H
- 2) MVI C, 03H
- 3) ADD B
- 4) ADD C
- 5) HLT

→ To subtract two values

MVI A, 05H

MVI B, 03H

SUB B

HLT

→ Load values and store Addition to another address

LDA 0000

MOV B, A

LDA 0001

ADD B

STA 0002

HLT

→ Load values and store subtraction to another address

LDA 0000

MOV B, A

LDA 0001

SUB B

STA 0002

HLT

OUTPUT

1) ADDING TWO VALUES

Registers			Address (Hex)	Address	Data
A	08		0000	0	6
BC	05	03	0001	1	5
DE	00	00	0002	2	14
HL	00	00	0003	3	3
PSW	00	00	0004	4	128
PC	00	07	0005	5	129
SP	FF	FF	0006	6	118
Int-Reg	00		0007	7	0

```
MVI B, 05H
MVI C, 03H
ADD B
ADD C
HLT
```

```
0000 0 6
0001 1 5
0002 2 14
0003 3 3
0004 4 128
0005 5 129
0006 6 118
0007 7 0
```

2) SUBTRACTING TWO VALUES

Registers			Address (Hex)	Address	Data
A	02		0000	0	62
BC	03	00	0001	1	5
DE	00	00	0002	2	6
HL	00	00	0003	3	3
PSW	00	00	0004	4	144
PC	00	06	0005	5	118
SP	FF	FF	0006	6	0
Int-Reg	00		0007	7	0

```
MVI A, 05H
MVI B, 03H
SUB B
HLT
```

```
0000 0 62
0001 1 5
0002 2 6
0003 3 3
0004 4 144
0005 5 118
0006 6 0
0007 7 0
```

3) LOAD VALUES AND STORE ADDITION TO ANOTHER ADDRESS

Registers			Address (Hex)	Address	Data
A	08		0000	0	5
BC	05	00	0001	1	3
DE	00	00	0002	2	8
HL	00	00	0003	3	0
PSW	00	00	0004	4	0
PC	03	F4	0005	5	0
SP	FF	FF			
Int-Reg	00				

```
LDA 0000
MOV B,A
LDA 0001
ADD B
STA 0002
HLT
```

```
0000 0 5
0001 1 3
0002 2 8
0003 3 0
0004 4 0
0005 5 0
```

4) LOAD VALUES & STORE SUBTRACTION TO ANOTHER ADDRESS

Registers			Address (Hex)	Address	Data
A	02		0000	0	3
BC	03	00	0001	1	5
DE	00	00	0002	2	2
HL	00	00	0003	3	0
PSW	00	00	0004	4	0
PC	03	F4			
SP	FF	FF			
Int-Reg	00				

```
LDA 0000
MOV B,A
LDA 0001
SUB B
STA 0002
HLT
```

```
0000 0 3
0001 1 5
0002 2 2
0003 3 0
0004 4 0
```